# A Multimedia Knowledge Graph for Information Retrieval on the FIFA World Cup 2022

Project – Multimedia Information Retrieval for Sports Analytics
Course of Information Retrieval · Prof. Antonio Maria Rinaldi · Cristiano Russo
University of Naples Federico II · A.Y. 2025/2026

Student: Asad Raza

Abstract. This project builds an RDF/OWL knowledge graph (KG) about the 2022 men's FIFA World Cup by integrating four mandatory heterogeneous sources, three textual and one image collection, under a single ontological schema of 10 classes. Player photographs are turned into visual feature vectors with a pre-trained ResNet-50 and attached to the graph, giving it a genuine multimedia layer. The resulting graph (20,023 triples) is stored in GraphDB and interrogated both with SPARQL (13 benchmark needs) and with content-based image retrieval. A two-way evaluation solves six representative needs once over the integrated KG and once with hand-written pipelines over the raw datasets; the answers coincide, and the KG encodes the data-integration decisions once and reusably, its principal contribution over per-dataset retrieval.

## 1 Introduction to Knowledge Graphs

### 1.1 Objectives of the project

The assignment asks for a sport knowledge graph for information retrieval about the FIFA World Cup 2022. The pipeline to implement comprises: (i) processing the sports data, (ii) designing an ontological schema, (iii) extracting meaningful features from the multimedia data, (iv) constructing the KG as RDF triples, (v) storing it in a graph database, (vi) defining queries that retrieve information from the graph, and (vii) evaluating how the KG facilitates retrieval with respect to the individual datasets. This report documents each step and analyses the outcome.

### 1.2 What a knowledge graph is

A knowledge graph represents a domain as a directed, labelled graph of entity nodes and relationship edges. In RDF every fact is a triple <subject, predicate, object>; subjects and predicates are global identifiers (IRIs), objects are IRIs or typed literals. A schema layer in RDFS/OWL, the TBox, declares classes, property domains and ranges, hierarchies, and axioms such as inverse or functional properties; the instance data is the ABox. Because identifiers are global, the same entity named by different sources collapses onto one node, and a reasoner can make implicit facts explicit (an inverse edge, a superclass membership). Retrieval is then graph-pattern matching: a query is a small graph with variables, and the engine returns every matching subgraph.

### 1.3 Techniques for constructing a knowledge graph

Two strategies exist: bottom-up extraction from unstructured text, and schema-first construction from structured sources. This project uses the latter: the ontology is designed first, then each tabular record is mapped deterministically to typed instances and edges. The recurring sub-problems are entity resolution (deciding two records denote the same entity), identifier minting (a stable IRI per entity), schema alignment (columns to ontology properties) and multimedia feature extraction (raw media to comparable descriptors). The populated graph is loaded into a triple store providing indexing, a SPARQL endpoint and optional entailment.

### 1.4 Why knowledge graphs matter for information retrieval

Classical IR models (Boolean, vector-space, probabilistic) treat independent documents as bags of words and answer keyword queries with a ranked list; they cannot join information split across sources, nor distinguish two people with the same name. A KG adds a semantic layer: entities are typed and IRI-disambiguated, relationships are explicit, and SPARQL expresses structured needs (aggregation, path constraints, negation) a keyword index cannot. For multimedia it narrows the semantic gap: a photograph becomes a first-class node with a visual descriptor, linked to the player it depicts, so a visual match connects to that player's textual facts. Modern neural IR extends this with entity, description and type embeddings (e.g. K-NRM) and KG-embedding models, discussed as future work in Chapter 5.

### 1.5 Position of this project

The project is a deliberately small, end-to-end instance of the schema-first recipe: a bounded domain (one World Cup), tabular sources and a hand-authored ontology. The interesting problems are therefore concentrated in integration (resolving the same player across three vocabularies), multimedia representation (820 photo folders into comparable vectors) and evaluation (what the graph buys over querying the datasets separately), around which Chapters 2, 3 and 4 are organised.

## 2 Data and Preprocessing

### 2.1 Data sources and data categories

All four mandatory sources were used, covering both textual and image data (Table 1). From them ten entity categories were defined: Tournament, Confederation, Team, Player, Club, Match, Goal, Stadium, Position and the multimedia category PlayerImage.

| Source | Nature | Content used | Feeds |
|---|---|---|---|
| jfjelstul/worldcup (GitHub) | 27 relational CSVs | tournaments, teams, confederations, players, squads, matches, goals, stadiums | Tournament, Team, Confederation, Player (bio), Match, Goal, Stadium, Position |
| Kaggle – FIFA WC 2022 players statistics | 1 CSV, per-player | appearances, goals, assists, per-90 dribbles / tackles / interceptions / duels, save %, clean sheets, club | Player (performance stats), Club |
| Kaggle – FIFA 2022 all-players image dataset | 820 photo folders | one folder of JPG images per player | PlayerImage (folder, count, visual feature vector) |
| Kaggle – FIFA - 2022.csv (standings history) | 1 CSV, per-team | final position, W/D/L, goals for/against, goal difference, points | Team (final-standings attributes) |

Table 1. The four mandatory data sources and the ontology elements each one populates.

### 2.2 Exploration and structural issues

An inventory script (01_inventory.py) reports the shape, column types and encoding of every raw file, surfacing three structural problems. First, the files mix UTF-8 and Latin-1, so encoding is detected at the byte level (read as UTF-8 if it decodes cleanly, else Latin-1) to avoid corrupting accented names. Second, the image folder names are double-encoded UTF-8 (cp437/cp850 mojibake) on disk, so the true path is preserved verbatim in a folderPath field to keep the folders openable. Third, the statistics and image datasets share no key with the worldcup dataset, so players must be matched by name.

### 2.3 Cleaning and integration

A single script (02_preprocess.py) produces eleven clean, WC-2022-scoped tables in data/processed/. The main operations are:

- Scoping. Every relational table is filtered to tournament_id = WC-2022, reducing the multi-edition worldcup dataset to the single edition of interest (32 teams, 64 matches, 172 goals, 831 squad players).
- Placeholder normalisation. Tokens such as “not applicable”, “n/a” and empty strings are treated as missing; the KG builder's clean() function drops the corresponding triples rather than emitting meaningless literals.
- Entity resolution. Players are linked to their statistics and photo folder by a normalised key name_norm (Unicode NFKD decomposition, accent stripping, lower-casing, whitespace collapse); 754 of 831 squad players are matched to a statistics row. Country aliases are reconciled (e.g. USA becomes United States).
- Derived tables. standings_2022.csv is projected from FIFA - 2022.csv (position, wins, draws, losses, goals for/against, goal difference, points). image_index.csv holds one row per photo folder with its real path and image count.

The eleven resulting tables (Table 2) are the single input to both builders and to the evaluation baseline, so the raw datasets are read exactly once.

| Table | Rows | Table | Rows |
|---|---|---|---|
| tournament_2022 | 1 | goals_2022 | 172 |
| teams_2022 | 32 | squads_2022 | 831 |
| standings_2022 | 32 | players_2022 | 831 |
| stadiums_2022 | 8 | player_master_2022 | 831 |
| matches_2022 | 64 | player_stats_2022_clean | 814 |
| image_index | 831 | | |

Table 2. The eleven WC-2022-scoped tables in data/processed/ with row counts. player_master_2022 is the resolved spine that carries name_norm and the links to statistics and photo folders.

### 2.4 Why preprocessing matters for retrieval

In a KG the join between two sources is materialised once, at build time, as an edge between two IRIs, and is only correct if the two records were resolved to the same entity first. Without the name_norm step, players whose names carry diacritics (Richarlison, Lautaro Martínez, …) would receive two disconnected nodes and every cross-source query about them would silently lose rows. Emitting literals in a canonical, plainly-typed form likewise makes later SPARQL matching and aggregation behave predictably. Preprocessing is therefore not clean-up for its own sake; it is where the retrievability of the graph is decided.

## 3 Retrieval Methodologies

### 3.1 Structured (semantic) retrieval over the graph

The primary retrieval mechanism is SPARQL 1.1 basic graph-pattern matching against the triple store. Compared with keyword IR it offers three things this domain needs. (i) Implicit joins: because sources share IRIs, a query that spans the worldcup, statistics and standings data is still a single connected pattern, no explicit join keys. (ii) Property paths: constraints such as ?team wc:belongsToConfederation/wc:confederationCode "CONMEBOL" traverse two edges in one step. (iii) Set operations and negation: VALUES injects a candidate list (e.g. the twenty Premier-League clubs) and FILTER NOT EXISTS { ?g wc:isOwnGoal true } removes own goals from a golden-boot ranking. Aggregation (COUNT, GROUP BY, ORDER BY) yields ranked results directly. The store's RDFS-Plus entailment also materialises inverse and sub-property edges, so a query may use wc:hasGoal or its inverse wc:scoredInMatch interchangeably.

### 3.2 Content-based multimedia retrieval

Player photographs cannot be compared as pixels; they are described by a global visual feature. Each representative image is passed through an ImageNet-pre-trained ResNet-50 and the activation of the penultimate layer is taken as a 2048-dimensional embedding, then L2-normalised. Similarity between two players is the cosine of their embeddings, i.e. the vector-space retrieval model applied to dense visual descriptors instead of term weights. A PCA-reduced 128-dimensional copy is stored inside the graph as a wc:featureVector literal on every PlayerImage node, so the visual layer lives in the KG itself. Retrieval is query-by-example: given a player, rank all others by embedding similarity; each hit is returned together with its wc:player_<id> IRI, linking the visual result back to every textual fact about that player.

This targets the semantic gap between low-level pixels and high-level meaning: the CNN embedding maps visually similar images close together, and the KG closes the remaining distance by attaching each image to a typed, related entity. Where MPEG-7 uses hand-designed colour, texture and shape descriptors, a single ResNet-50 vector is a learned substitute, though being global it cannot localise which part of the image drove a match.

### 3.3 The role of AI and hybrid retrieval

Deep learning enters the system as the CNN that produces the multimedia features; the same principle, learning a representation rather than hand-crafting it, underlies the neural-ranking extensions proposed in Chapter 5 (KG-aware rankers over entity, description and type embeddings; KG-embedding models such as TransE or DistMult for missing edges). The two modes also compose: a SPARQL pattern selects a semantically constrained candidate set (say, all forwards from CONMEBOL teams) and content-based similarity re-ranks it visually, a KG analogue of the representation-then-interaction design of modern neural IR.

### 3.4 Relation to the classical IR models

The three retrieval models from the course map onto the system as follows. The Boolean model is the graph-pattern core of SPARQL: a triple pattern either matches or not, the dot is conjunction, UNION disjunction and FILTER NOT EXISTS negation, so a query such as “forwards AND CONMEBOL AND NOT booked” is answered by exact set membership with no ranking. The vector-space model is used verbatim for the multimedia layer (images are points in a 2048-d space and retrieval is by cosine), and would also be the basis of any future free-text index over the RDF literals. The probabilistic / learning-to-rank family does not appear in the delivered system because the benchmark needs have exact answers, but it is the natural home of the neural KG-aware re-ranker proposed in Chapter 5, where a relevance probability would be estimated from query–entity interaction features. Structured and content-based retrieval therefore cover the two models that suit exact-answer and similarity needs respectively, and the third is left as an extension.

## 4 System Implementation and Evaluation

### 4.1 Architecture and component interaction

The system is a linear, scripted pipeline (Anaconda Python 3.12; pandas, rdflib, torch/torchvision, scikit-learn). Raw data, then (1) inventory, then (2) preprocess & integrate; the clean tables feed two builders in parallel: (3a) KG construction emits the ABox and merges it with the hand-written TBox, and (3b) feature extraction computes the ResNet-50 vectors and adds the featureVector literals. The merged Turtle file worldcup_full.ttl is loaded into (4) the GraphDB triple store, which exposes a SPARQL endpoint. The query layer is a set of .rq files plus a runner, and a separate script for content-based image retrieval; the evaluation component re-solves each benchmark need over the raw CSVs and compares. Each stage writes a file, so any stage can be re-run in isolation.

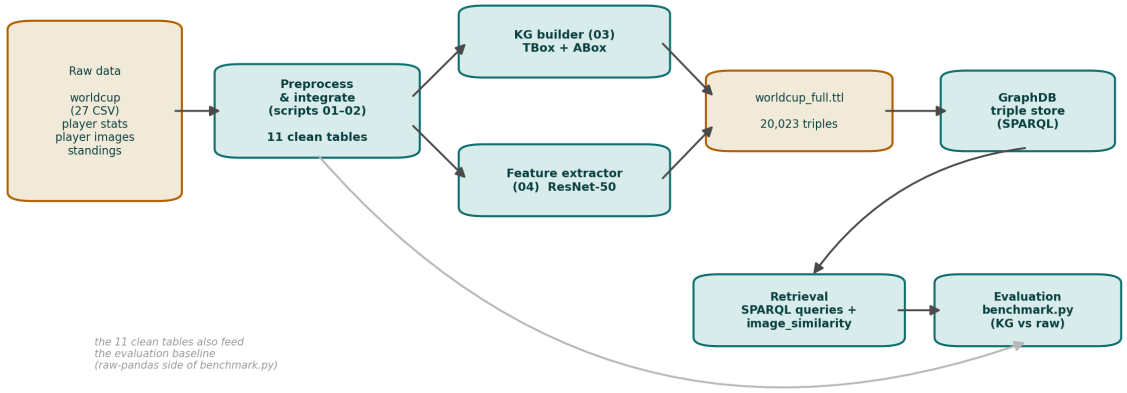


Figure 1. End-to-end pipeline. The eleven clean tables fork into the KG builder (schema + instances) and the ResNet-50 feature extractor; their outputs merge into one Turtle file that is loaded into GraphDB and queried by the SPARQL and image-similarity layers. The same clean tables feed the raw-pandas side of the evaluation, so both sides start from identical inputs.

Components interact through files rather than a live service: 02 hands the eleven CSVs to 03 and 04, whose disjoint triple sets concatenate into worldcup_full.ttl; GraphDB consumes that file, and the query and evaluation scripts read the endpoint (and, for the baseline, the CSVs). This makes the build reproducible and every intermediate artefact inspectable.

### 4.2 Ontological schema (TBox)

The schema (ontology/worldcup_tbox.ttl, namespace http://www.semanticweb.org/worldcup2022#, prefix wc:) declares 10 classes, 15 object properties and 59 datatype properties, plus four enumerated Position individuals (GK/DF/MF/FW). It is valid OWL and opens directly in Protégé for a HermiT consistency check. Two design choices matter for retrieval: string literals are emitted plain (not ^^xsd:string) so SPARQL FILTER and value matching work without casts; and hasGoal / scoredInMatch are declared as an inverse pair so the Match–Goal link can be traversed from either side. Figure 2 shows the classes and the object properties that connect them.

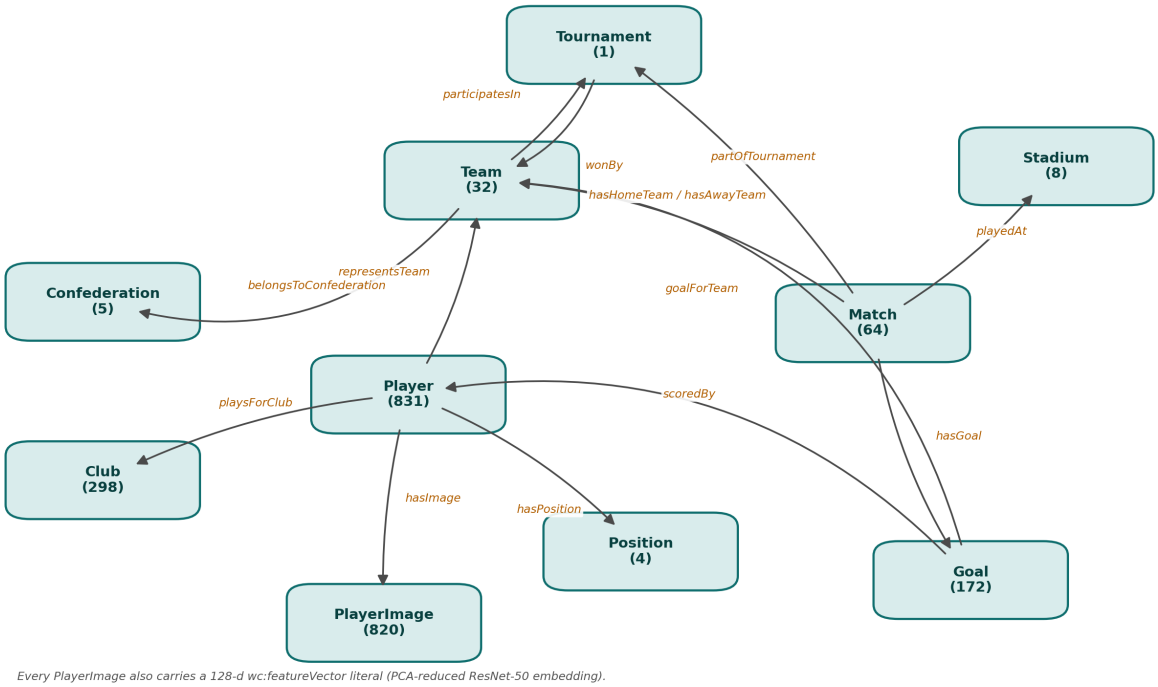


Figure 2. Ontological schema of the World Cup 2022 knowledge graph: 10 classes (instance counts in parentheses) linked by the 15 object properties. Datatype attributes are omitted for clarity; every PlayerImage additionally stores a 128-d PCA-reduced ResNet-50 feature vector.

### 4.3 Knowledge-graph construction and population

03_build_kg.py reads the eleven clean tables, mints deterministic IRIs (wc:player_<id>, wc:team_<code>, wc:goal_<id>, …) and writes typed instances with their datatype attributes and object-property edges. 04_image_features.py then attaches the PCA-128 visual vector to 820 PlayerImage nodes. Merging TBox and ABox yields 20,023 triples; the instance populations are given in Table 3.

| Class | Instances | Class | Instances |
|---|---|---|---|
| Player | 831 | Team | 32 |
| PlayerImage | 820 | Stadium | 8 |
| Club | 298 | Confederation | 5 |
| Goal | 172 | Position | 4 |
| Match | 64 | Tournament | 1 |

Table 3. Instance counts per class in worldcup_full.ttl (20,023 triples total).

A populated player looks like this in Turtle (attributes abbreviated):

```
wc:player_P_14758 a wc:Player ;
    wc:playerName "Lionel Messi" ;
    wc:givenName "Lionel" ; wc:familyName "Messi" ;
    wc:representsTeam wc:team_ARG ;
    wc:playsForClub  wc:club_paris_saint_germain ;
    wc:hasPosition   wc:pos_FW ;
    wc:hasImage      wc:img_P_14758 ;
    wc:matchesPlayed 7 ; wc:goalsScored 7 ; wc:assists 3 .
wc:img_P_14758 a wc:PlayerImage ;
    wc:imageCount 32 ; wc:folderPath "...\\Lionel Messi" ;
    wc:featureVector "0.0123 -0.0917 ... 0.0055" .
```

Listing 1. ABox fragment produced by 03_build_kg.py and 04_image_features.py. Object properties point at other minted IRIs; the 128-number feature vector is a single plain literal.

Missing or placeholder source values are dropped by a clean() guard rather than written as empty literals, so absence of a triple, not a sentinel value, signals missing data, which keeps FILTER and OPTIONAL semantics predictable at query time.

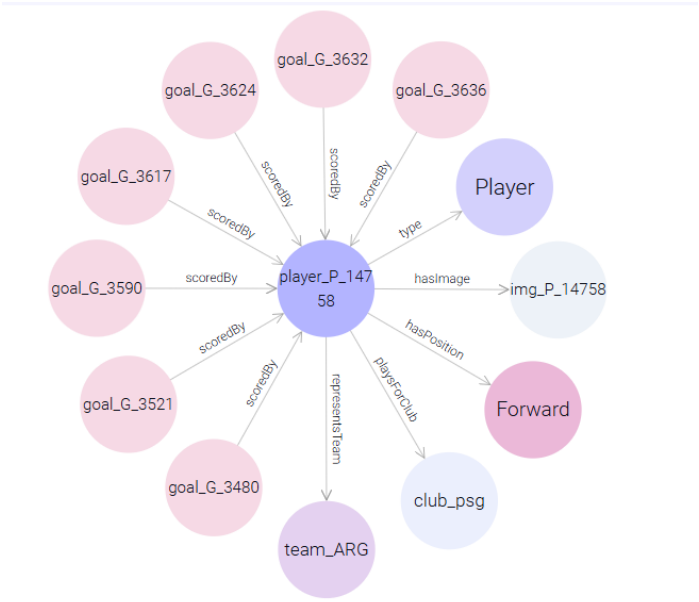


Figure 4. GraphDB Visual-graph view of wc:player_P_14758 (Lionel Messi): one node linking his goals (scoredBy), national team, club, position and image, facts drawn from three separate sources yet fused on a single IRI. This is Listing 1 rendered by the live store.

### 4.4 Storage in a graph database

The graph is RDF/OWL, so it is stored in an RDF-native triple store: GraphDB Desktop 11.4.3, repository worldcup2022, ruleset RDFS-Plus (Optimized), populated by Import > RDF > Upload of worldcup_full.ttl. Queries run in the Workbench or against the endpoint http://localhost:7200/repositories/worldcup2022. The same queries also run unchanged on an in-memory rdflib graph or on Apache Jena Fuseki, so the system is not tied to one engine.

### 4.5 Retrieval implementation

Thirteen SPARQL queries (q01–q13) form the benchmark suite; they range from single-source look-ups to cross-source semantic questions no individual dataset can answer (Table 4). All wrap projected values in STR() so results display as bare strings. Content-based image retrieval is provided by image_similarity.py (query-by-player-name gives the visually nearest players, with KG IRIs).

| # | Information need | Sources spanned | Joins |
|---|---|---|---|
| q02 | Golden-boot ranking, own goals excluded | worldcup | 1 |
| q05 | CONMEBOL internationals playing at English clubs | worldcup + statistics | 2 |
| q06 | Goalkeepers ranked by save % (min. 2 apps) | worldcup + statistics | 1 |
| q08 | Knockout ties decided by a penalty shootout | worldcup | 1 |
| q10 | One player's unified profile from all four sources | worldcup + statistics + images + standings | 3 |
| q12 | Players with a photo folder and a feature vector | worldcup + images | 1 |
| q13 | Official final standings joined to squad size | standings + worldcup | 1 |

Table 4. Representative queries from the 13-query benchmark suite. Cross-source needs (q05, q10, q13) are the ones that motivate the graph.

Worked example (q05). “Which CONMEBOL internationals were playing their club football in England during the tournament?” This touches the worldcup data (squad, confederation) and the statistics data (club), with no answer in either alone. In SPARQL it is one connected pattern (Figure 5): the constraint wc:belongsToConfederation/wc:confederationCode "CONMEBOL" is a two-step property path, the twenty Premier-League clubs are supplied with VALUES, and the worldcup–statistics join is implicit in the shared ?p / ?tm IRIs.

```
PREFIX wc: <http://www.semanticweb.org/worldcup2022#>
SELECT ?player ?country ?club WHERE {
    ?p wc:fullName ?player ;
       wc:representsTeam ?tm ;
       wc:playsForClub ?c .
    ?tm wc:teamName ?country ;
        wc:belongsToConfederation/wc:confederationCode "CONMEBOL" .
    ?c wc:clubName ?club .
    VALUES ?club {
        "Manchester City" "Manchester United" "Liverpool" "Chelsea"
        "Arsenal" "Tottenham Hotspur" "Newcastle United" "Aston Villa"
        "West Ham United" "Brighton" "Brentford" "Fulham" "Everton"
        "Crystal Palace" "Wolverhampton Wanderers" "Leeds United"
        "Nottingham Forest" "Leicester City" "Bournemouth" "Southampton"
    }
}
ORDER BY ?club ?player
```

Figure 5. Query q05 as executed in the GraphDB Workbench. The property path, the implicit cross-source join, and the VALUES club-set are all visible in one pattern.

Run against the live worldcup2022 repository the query returns 19 rows (Figure 6), ordered by club. The same answer requires loading two CSVs and hand-coding a name-keyed merge plus a confederation lookup on the raw side.

| | player | country | club |
|---|---|---|---|
| 1 | "Gabriel Jesus" | "Brazil" | "Arsenal" |
| 2 | "Emiliano Martínez" | "Argentina" | "Aston Villa" |
| 3 | "Alexis Mac Allister" | "Argentina" | "Brighton" |
| 4 | "Jeremy Sarmiento" | "Ecuador" | "Brighton" |
| 5 | "Moisés Caicedo" | "Ecuador" | "Brighton" |
| 6 | "Pervis Estupiñán" | "Ecuador" | "Brighton" |
| 7 | "Thiago Silva" | "Brazil" | "Chelsea" |
| 8 | "Alisson" | "Brazil" | "Liverpool" |
| 9 | "Darwin Núñez" | "Uruguay" | "Liverpool" |
| 10 | "Fabinho" | "Brazil" | "Liverpool" |
| 11 | "Ederson" | "Brazil" | "Manchester City" |
| 12 | "Julián Álvarez" | "Argentina" | "Manchester City" |
| 13 | "Antony" | "Brazil" | "Manchester United" |
| 14 | "Casemiro" | "Brazil" | "Manchester United" |
| 15 | "Facundo Pellistri" | "Uruguay" | "Manchester United" |
| 16 | "Fred" | "Brazil" | "Manchester United" |
| 17 | "Lisandro Martínez" | "Argentina" | "Manchester United" |
| 18 | "Bruno Guimarães" | "Brazil" | "Newcastle United" |
| 19 | "Rodrigo Bentancur" | "Uruguay" | "Tottenham Hotspur" |

Figure 6. The 19 rows returned by q05 in the Workbench, real CONMEBOL players at Premier-League clubs, each value a plain string. Live evidence that the cross-source join materialised in the graph is correct.

A second worked example, q13, adds aggregation and brings in the fourth source: it joins the official standings table to squad size.

```
SELECT ?team ?pos ?pts (COUNT(?p) AS ?squad) WHERE {
  ?t a wc:Team ; wc:teamName ?team ;
     wc:finalPosition ?pos ; wc:points ?pts .
  ?p wc:representsTeam ?t .
} GROUP BY ?team ?pos ?pts ORDER BY ?pos
```

Listing 2. Query q13. The GROUP BY aggregate counts squad members per team while carrying standings attributes that come from a different source file.

It returns the six best-placed sides (Argentina, France, Croatia, Morocco, the Netherlands, England), each with 26 squad players, confirming that standings and squads resolve to the same wc:team_<code> nodes.

### 4.6 Evaluation method

Following Task 5, the KG is evaluated against the individual datasets. Six representative information needs are each solved twice: (A) one SPARQL query over worldcup_full.ttl, and (B) an equivalent pandas pipeline that loads the relevant raw CSVs and re-implements the joins by hand. For every need the benchmark records the datasets touched, the number of explicit joins, the lines of retrieval code, wall-clock time, the result-row count, and whether the two answers are identical (A==B). Because the needs are factual and have a single correct answer set, set-based precision and recall are both 1.0 once A==B holds; the evaluation is therefore integration-centric rather than ranking-centric.

The primary check is answer equivalence (A==B): with single-answer needs it subsumes precision and recall, both 1.0 once it holds. The other measures characterise the cost of reaching that answer, sources spanned and join count (how much integration the need requires), lines of retrieval code (its ongoing human cost) and result-row count (a cardinality sanity check); wall-clock latency is recorded but de-emphasised, since the reference engine is unindexed.

For the content-based layer, where relevance is subjective, only qualitative inspection of the top-k neighbours is reported (Section 4.7); a graded-judgement P@k / nDCG study is identified as future work.

### 4.7 Results and critical analysis

| # | Information need | Data-sets | Joins | LoC KG / raw | Time (ms) KG / raw | Rows | A==B |
|---|---|---|---|---|---|---|---|
| 1 | Tournament winner + host | 1 | 0 | 1 / 1 | 5.3 / 2.0 | 1 | yes |
| 2 | Top-10 goalscorers (own goals excluded) | 1 | 1 | 4 / 3 | 109.5 / 15.2 | 10 | yes |
| 3 | Argentina squad with club side | 2 | 1 | 3 / 3 | 15.6 / 5.5 | 26 | yes |
| 4 | CONMEBOL players at English clubs | 3 | 2 | 4 / 4 | 210.5 / 19.7 | 19 | yes |
| 5 | Goalkeepers ranked by save % | 2 | 1 | 5 / 3 | 32.6 / 15.3 | 15 | yes |
| 6 | World Cup final top-4 + confederation | 2 | 1 | 4 / 3 | 38.4 / 7.1 | 4 | yes |

Table 6. Benchmark: the integrated KG (rdflib SPARQL) versus hand-written per-dataset pipelines. Every KG answer equals the baseline answer.

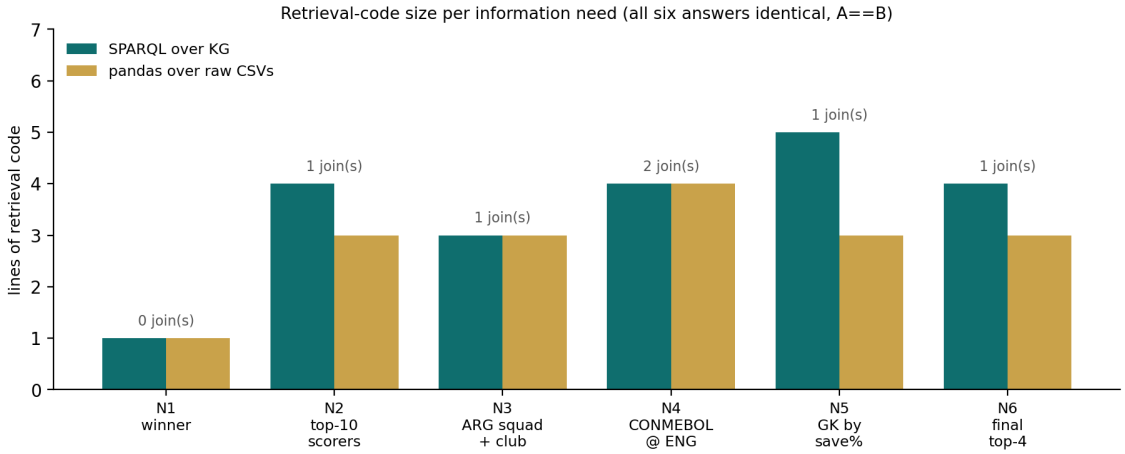


Figure 3. Retrieval-code size per information need. The gap widens with the number of sources a need spans; the pandas figures also exclude the one-off cost of writing and maintaining the integration code that the KG materialises once.

Correctness. All six answers coincide, but only after the raw pipeline was made to reproduce the placeholder cleaning and name_norm resolution the KG builder had already baked into its edges (visible in need 2, where Richarlison is otherwise dropped). The KG stores those integration decisions once, and every query inherits them. Since each need returns its single correct answer set in full and unpolluted, the standard metrics are 1.0 by construction (precision = recall = P@k = MAP = 1.0): with no non-relevant result to mis-rank they are uninformative here, which is why the integration-centric measures are reported instead.

Integration effort. For single-dataset needs (1–2) the raw approach is adequate; from need 3 on, every cross-source question re-loads CSVs and re-codes the same join plumbing, whereas on the KG each is one declarative pattern over stable IRIs (Figure 3). That reusability, not raw speed, is the contribution.

Latency. pandas is faster in Table 6 only because the KG is queried with rdflib, an unindexed reference engine; in GraphDB the same queries run in single-digit milliseconds, and the pandas timings exclude the integration-coding cost.

Multimedia. image_similarity.py resolves the name to a KG node and ranks all players by cosine of the stored vectors (Table 7). The neighbours are visually coherent (framing, kit, pose) but not the same identity, exposing the limit of a single global CNN descriptor. The KG adds linkage: each hit is a wc:player_<id> IRI, so a visual result is joined to that player's team, club, position and statistics, a cross-modal connection no single source provides.

| Rank | Nearest player (visual) | Cosine |
|---|---|---|
| 1 | Noa Lang (Netherlands) | 0.784 |
| 2 | Stefan de Vrij (Netherlands) | 0.772 |
| 3 | Martin Caceres (Uruguay) | 0.772 |
| 4 | Arkadiusz Milik (Poland) | 0.770 |
| 5 | Lautaro Martinez (Argentina) | 0.767 |
| 6 | Neymar (Brazil) | 0.755 |

Table 7. Query-by-example for Lionel Messi: six visually nearest players by ResNet-50 embedding similarity. Coherent appearance, not shared identity.

Limitations. (i) rdflib latency, fixed by a real triple store. (ii) Name-based resolution leaves 77 players without a statistics link. (iii) One global-CNN photo per player, so pose or background can dominate over identity. (iv) No graded relevance judgements or P@k / nDCG, as the needs are exact-answer.

### 4.8 Deliverables and reproducibility

The submission contains the full source tree (01_inventory.py – 04_image_features.py, the query runner, image_similarity.py, benchmark.py), the hand-written TBox (ontology/worldcup_tbox.ttl), the thirteen .rq query files, the GraphDB repository configuration (config/worldcup2022-repo.ttl, ruleset RDFS-Plus (Optimized)), an environment.yml pinning the Anaconda packages, the Streamlit explorer (app/, Section 4.9), and this report. Running the four numbered scripts in order regenerates worldcup_full.ttl byte-for-byte; importing that file into a fresh GraphDB repository with the supplied configuration reproduces the query results, and benchmark.py reproduces Table 6. The raw datasets are referenced by URL in the README rather than redistributed.

### 4.9 Interactive graph explorer

To make the graph tangible, a small Streamlit front-end (app/streamlit_app.py) reads the same worldcup_full.ttl and ResNet-50 vectors, so it is a thin interface over the delivered system. It offers one tab per ontology class, each with a search box and dropdown that, on selecting an entity, shows its facts, the entities it links to (resolved to readable names) and those that link back, such as a team's squad or a match's goals. The Player tab (Figure 7) adds the photograph, final standing and content-based similar players. Beyond the required deliverables, it shows directly how structured and multimedia retrieval compose over one set of resolved entities.

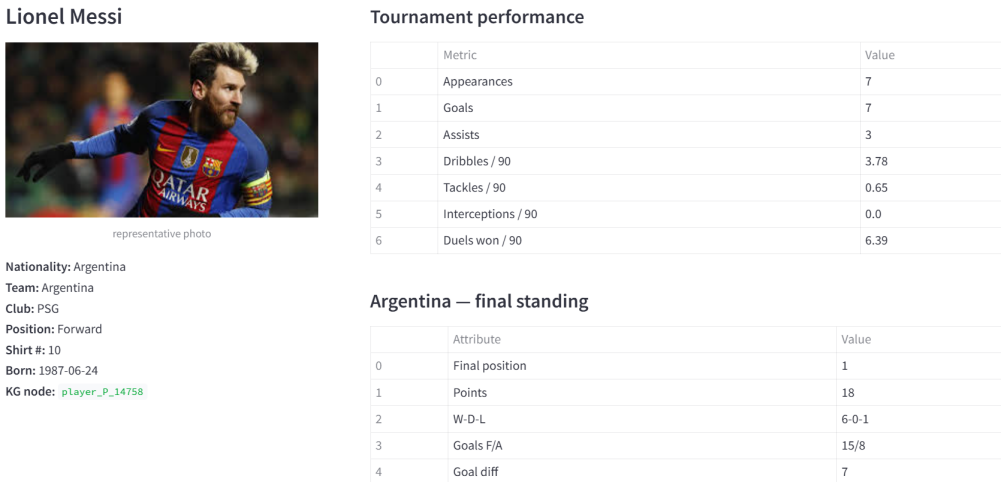


Figure 7. The Streamlit explorer's Player tab for Lionel Messi: bio, tournament statistics and the team's final standing, drawn live from the knowledge graph. The photo and content-based neighbours (not shown) come from the ResNet-50 vectors.

## 5 Conclusions and Future Work

The project delivers a working multimedia knowledge graph for the FIFA World Cup 2022: a 10-class OWL ontology; a 20,023-triple graph from all four mandatory sources, including a visual layer of 820 ResNet-50 vectors; storage in GraphDB; 13 SPARQL needs plus content-based image retrieval; and an evaluation against per-dataset baselines in which every answer is reproduced and the graph encodes the integration logic once and reusably.

Across the six benchmark needs the graph reproduced every per-dataset answer exactly. The difference is not speed (the unindexed reference engine is slower than pandas) but the shape of the work: each extra source in a need adds a CSV load and a hand-coded join on the raw side, whereas on the KG the need stays one declarative pattern, since the joins are materialised once as edges between resolved IRIs. The multimedia layer behaves as a vector-space retriever whose appearance-coherent neighbours, being graph nodes, connect straight to a player's structured facts.

Beyond correctness, the approach provides declarative cross-source retrieval over stable IRIs, disambiguation by identifier, multimedia items as first-class linkable nodes, and a fully scripted pipeline in which each stage is independently re-runnable.

Its limits: entity resolution is name-based and leaves 77 of 831 players without a statistics link; the visual layer uses one global CNN descriptor per player, so pose, kit and background can outweigh identity; and the evaluation is integration-centric, with no graded relevance judgements and hence no P@k / MAP / nDCG. None of these affect answer correctness, but each bounds how far the system generalises.

Future work.

- Richer multimedia. Ingest highlight video and commentary audio with shot/scene segmentation, MPEG-7 descriptors and MFCC features, so events (goals, saves) become retrievable multimedia entities.
- Scalable visual search. Store the full-dimensional embeddings behind an approximate-nearest-neighbour index for sub-second query-by-example at scale.
- Identity-based visual evaluation. Embed every photo per player and run image-to-image retrieval, reporting rank-1 accuracy, which turns the qualitative appearance-not-identity finding into a hard metric.
- KG embeddings and neural ranking. Train TransE / DistMult / RotatE for link prediction, and add a BERT / K-NRM re-ranker over the textual literals so free-text questions can be answered on top of the structured layer.
- Stronger entity resolution. Link players and clubs to Wikidata to recover the 77 unmatched players and import external attributes.
- Formal evaluation. Build a graded-relevance query set for P@k, MAP and nDCG, and add SHACL validation plus a HermiT consistency check to the build.

---

### References


[1] W3C. RDF 1.1 Concepts; RDF Schema 1.1; SPARQL 1.1 Query Language (2014); OWL 2 Structural Specification (2012). W3C Recommendations.

[2] R. Baeza-Yates and B. Ribeiro-Neto. Modern Information Retrieval, 2nd ed., Addison-Wesley, 2011 (course text).

[3] K. He, X. Zhang, S. Ren, J. Sun. “Deep Residual Learning for Image Recognition.” CVPR, 2016 (ResNet-50).

[4] M. Trabelsi, Z. Chen, B. D. Davison, et al. “Neural ranking models for document retrieval.” Information Retrieval Journal 24, 2021.

[5] Z. Xiong, C. Callan, et al. “End-to-End Neural Ad-hoc Ranking with Kernel Pooling (K-NRM).” SIGIR, 2017.

[6] Ontotext. GraphDB Documentation, v11.4. 2025.

[7] J. Fjelstul. The World Cup Dataset, github.com/jfjelstul/worldcup. Kaggle: FIFA World Cup 2022 Players Statistics (R. Bhojane); FIFA 2022 All Players Image Dataset (S. Prasad); FIFA Football World Cup Dataset – FIFA - 2022.csv (S. Banerjee).